A Comprehensive Security Framework for Autonomous AI Agents: Integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via protocols like the Model Context Protocol (MCP), they introduce a critical "Agency Gap." Traditional security perimeters are insufficient for protecting against prompt injection, semantic drift, and future cryptographic threats. This paper proposes a unified, three-tier security framework designed to protect agent-based systems across three dimensions: Space, Behavior, and Time. First, we establish *Structural Guardrails* (Space) using AST-based static analysis and secure sandboxing to contain malicious code execution. Second, we implement *Behavioral Zero-Trust* (Behavior) monitoring using a Mamba-based Sentinel to detect anomalous reasoning in real-time and enforce workload identity. Third, we ensure *Temporal Sovereignty* (Time) by integrating Post-Quantum Cryptography (PQC) and remote attestation to guarantee long-term integrity and resilience against future quantum adversaries. Our evaluation demonstrates that this "Triple-Lock" architecture provides robust, multi-layered defense with minimal computational overhead, maintaining agent responsiveness while ensuring system-wide security.**

***Index Terms*—AI Agents, Model Context Protocol (MCP), Agentic Security, Zero Trust, Post-Quantum Cryptography, Mamba, Guardrails, Sandbox.**

I. Introduction

The transition of Large Language Models (LLMs) to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can interact directly with host operating systems, manage sensitive data, and orchestrate complex API workflows. However, this "Agency" capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against "Prompt Injection" attacks, where an attacker manipulates the LLM's goal-oriented reasoning to execute unauthorized system commands [7].

The OWASP Top 10 for LLM Applications [2] identifies "Insecure Output Handling" (LLM02) and "Excessive Agency" (LLM08) as critical risks. Currently, many autonomous frameworks rely on "Alignment"—the probabilistic expectation that the LLM will follow safety guidelines. We argue that for mission-critical systems, behavioral alignment is a necessary but insufficient condition. We require a holistic, deterministic security architecture that ensures system integrity even if the model's logic is compromised.

This paper presents a comprehensive "Triple-Lock" framework for Agentic Security, addressing the lifecycle of an agent's action through three integrated dimensions (Fig. 1):

1) **Structural Containment (Space):** Establishing deterministic safety boundaries using AST inspection and OS-level sandboxing (Bubblewrap) to ensure "Secure-by-Default" tool execution.
2) **Behavioral Integrity (Behavior):** Implementing a Zero Trust Architecture (ZTA) where every action is verified by a verifiable workload identity and monitored by a Mamba-based "Reasoning Sentinel" to detect semantic anomalies.
3) **Temporal Sovereignty (Time):** Protecting against future cryptographic threats (Shor's algorithm) using Post-Quantum Cryptography (PQC) to ensure the long-term confidentiality and non-repudiability of agent history.

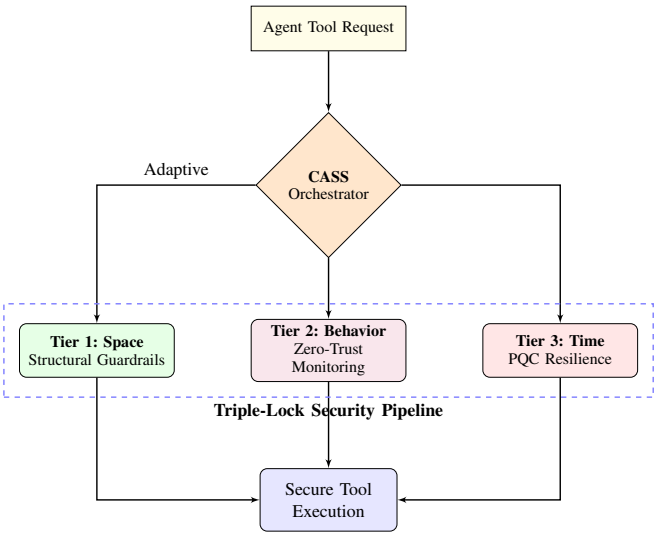

Fig. 1. Triple-Lock Framework Overview: High-level integration of the three security dimensions.

II. Related Work

Existing efforts to secure agents generally fall into two categories: *Behavioral Filtering* (e.g., Guardrails AI) and *Isolation Patterns* (e.g., AgentDojo [9]). While effective, behavioral filters often introduce high latency and are susceptible to the same reasoning failures as the agent they monitor. Isolation patterns provide containment but often lack continuous identity verification.

Our framework aligns with the **NIST Zero Trust principles** (SP 800-207) [3], treating every tool call as an untrusted workload. Furthermore, we address the "Store Now, Decrypt Later" (SNDL) threat by adopting NIST-standardized PQC primitives [5], [6], ensuring that agentic reasoning traces remain private in the post-quantum era.

III. Threat Model

We define four primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** Manipulating the agent's context to execute unauthorized commands or exfiltrate secrets [8].
2) **Semantic Drift/Malicious Intent:** An agent performing syntactically valid but semantically harmful actions (e.g., deleting a database file represented as a "document").
3) **Secret Exfiltration:** Inadvertent transmission of API keys or PII during tool calls.
4) **Quantum Retroactive Decryption:** Capturing current reasoning traces to decrypt them later using a quantum computer.

IV. Tier 1: Structural Guardrails (Space)

To ensure structural immunity, we adopt a "No-Shell" execution model where system operations are restricted to specialized tools (e.g., `execute_python`).

A. AST Inspection and Static Analysis

Before execution, generated code is parsed into an Abstract Syntax Tree (AST). A whitelist-based scanner blocks dangerous modules (e.g., `os, socket, pty`) and built-in functions (e.g., `eval, exec`). Specifically, it ensures that `subprocess` calls never use `shell=True`, and detects reflection-based attacks (e.g., `__subclasses__`, `globals()`) to enforce a strict separation between executable and arguments.

B. Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace via path resolution and symbolic link verification. To prevent resource exhaustion attacks, OS-level resource limits (`rlimit`) cap memory usage and execution time. Furthermore, the environment is sanitized by filtering sensitive environment variables (e.g., API keys) before passing them to the sub-process.

C. Sandboxing with Bubblewrap

The final containment is provided by **Bubblewrap** (`bwrap`), which utilizes Linux namespaces to create an unprivileged sandbox. Specifically, the implementation mounts the minimum read-only set of system paths required for Python execution (`/usr`, `/lib`, `/lib64`, `/bin`, `/sbin`) and provides an isolated `/tmp` via `tmpfs`. The `--unshare-all` flag enforces full network, IPC, UTS, and PID namespace isolation, eliminating all outbound network access and host process visibility. In high-risk scenarios, CASS escalates the sandbox profile to "isolated," switching the workspace bind-mount from read-write (`--bind`) to read-only (`--ro-bind`) for scripts that only require analysis.

V. Tier 2: Behavioral Zero-Trust (Behavior)

Identity ensures *who* is calling the tool, while monitoring ensures *why*.

A. Workload Identity and RBAC

Each agent instance is assigned a unique key pair. Every MCP tool call is accompanied by a signed **Identity Token** (JWT) containing assigned Role-Based Access Control (RBAC) roles. Remote servers verify this token to ensure the request originated from a legitimate, untampered agent and enforce strict tool access policies decoupled from the LLM's prompt. In Tier 3, this identity is upgraded to a PQC-hybrid signature. Crucially, public keys and remote attestation manifests are distributed via an Out-of-Band (OOB) trusted channel (e.g., MDM or Secure Enclave). This architectural choice completely avoids the Trust-On-First-Use (TOFU) vulnerability[1] and eliminates the need for external verification servers, maintaining the CLI tool's self-contained nature and resilience against initial compromise.

B. The Mamba Sentinel: Overcoming Dual-LLM Latency

Current agentic security frameworks often rely on a "Dual-LLM" approach (using a secondary model to verify the intent of the primary model). However, our empirical testing demonstrates that this inherently introduces a massive $O(N^2)$ Transformer overhead, causing severe User Experience (UX) degradation (frequently adding multi-second latency per tool call). To resolve this, we transition from the Dual-LLM intent analyzer (retained only as a legacy option) to a **Mamba (State Space Model)** [4] implementation as the default monitoring engine. Running locally with pure NumPy, Mamba monitors the agent's reasoning tokens in real-time with $O(N)$ linear scaling. By calculating a "Surprise Score" (cross-entropy loss), the Sentinel detects statistical anomalies correlating with "Jailbreaks" or adversarial intent in sub-millisecond timeframes. To minimize false positives during the early stages of agent deployment, we implement a **Self-Calibrating Sentinel**

[1]While the core protocol is designed for OOB provisioning to eliminate TOFU, the provided standalone reference implementation includes an optional bootstrap mode for environments lacking enterprise distribution infrastructure. This ensures immediate usability while allowing security-hardened deployments to disable TOFU via environment-specific configuration.

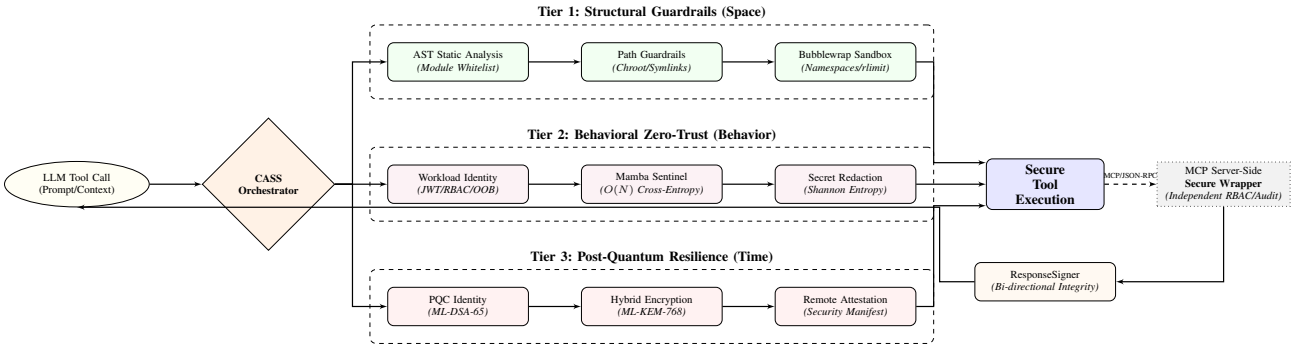

Fig. 2. Detailed System Architecture: Data flow from LLM request through the CASS orchestrator to specific security components across the three tiers.

where anomaly thresholds are dynamically derived from the Exponential Moving Average (EMA) of the model's own loss, ensuring the security baseline evolves alongside the agent's behavioral maturity. Furthermore, we employ a **High-Entropy Secret Detection** heuristic using Shannon Entropy to identify and redact API keys or PII instantly, regardless of the Sentinel's training state.

C. Bi-directional Verification with ResponseSigner

To ensure the integrity of the entire tool execution loop, we implement **Bi-directional Verification**. While the agent signs the tool request via an Identity Token, high-risk write tools (`edit_file`, `create_or_overwrite_file`) embed a **ResponseSigner** PQC signature (ML-DSA) in their return value, binding the response content to a unique `verification_id`. The `tool_executor` layer verifies this signature before passing the result to the LLM, preventing "Man-in-the-Middle" manipulation of tool outputs and ensuring that the model's subsequent reasoning is grounded in untampered observations.

Furthermore, when the Mamba Sentinel raises a **RED** alert on a high-risk tool, CASS activates "strict enforcement." Rather than autonomously blocking execution—which risks UX degradation and opaque failures—the system mandates a **Human-in-the-Loop (HITL)** approval dialog, surfacing the anomaly score and requiring explicit human confirmation. This design is consistent with the NIST AI Risk Management Framework (AI RMF) principle of maintaining human oversight over consequential AI actions.

VI. Tier 3: Post-Quantum Resilience (Time)

To protect the "Temporal Dimension," we integrate PQC primitives.

A. Quantum-Safe Identity and Hybrid Encryption

We upgrade the workload identity using **ML-DSA-65** (FIPS 204). Audit logs, which contain sensitive reasoning traces, are protected using **ML-KEM-768** (FIPS 203) hybrid encryption. Our implementation supports **PQC Agility**, dynamically switching between ML-DSA-44, 65, and 87 variants based on the risk level determined by CASS. As described in Section VII, three physically separate key pairs are provisioned on disk to satisfy cryptographic isolation across security tiers.

B. PQC Remote Attestation and Audit Continuity

The client generates a SHA-256 manifest of its core security code (covering six critical files including `identity.py`, `policy.py`, and `audit.py`), signed with an ML-DSA private key. This allows the server to verify that the agent is running on an "Authentic Stack" where Tier 1 guardrails are active. In strict mode (`LLM_CLI_STRICT_SECURITY=1`), the absence of a pre-provisioned manifest causes startup to abort, eliminating Trust-On-First-Use (TOFU) risk in production deployments.

To ensure the tamper-evidence of long-running logs, we implement **Audit Chain Continuity** using a combination of two mechanisms: (1) on log rotation, overflowing entries are written to a timestamped archive file and a signed *snapshot anchor* entry is prepended to the remaining log, recording both the `snapshot_prev_hash` and `snapshot_first_hash` to maintain chain verifiability across file boundaries; (2) a binary **Merkle Tree** is computed over all log entries, producing a single Merkle Root that can be externally anchored (e.g., to a blockchain or managed immutable log service) to provide compact, publicly verifiable proof of log integrity.

VII. System Architecture and Integration

To synthesize the three tiers into a cohesive "Triple-Lock" architecture, we introduce **Context-Adaptive Security Scaling (CASS)**. CASS serves as the central orchestration engine, dynamically integrating the spatial, behavioral, and temporal defenses based on the specific risk profile of each tool execution request.

The "Triple-Lock" architecture operates as a unified pipeline governed by CASS (Fig. 2). When an agent requests a tool call: 1. **Risk Assessment (CASS):** The system evaluates the requested tool (e.g., `search_web` vs. `execute_python`) to determine the required security posture. 2. **Tier 1 - Structural Containment:** The **Guardrail Manager** applies AST analysis, enforces path/resource limits, and prepares the unprivileged Bubblewrap sandbox. 3. **Tier 2 - Behavioral Verification:** The **Zero-Trust Controller** verifies the RBAC-embedded Identity Token and computes the Mamba Surprise Score. The Multi-Signal detector redacts any identified high-entropy secrets. 4. **Tier 3 - Temporal Anchoring:** For high-risk operations, the **PQC Module** signs the request with a

Hybrid-PQC token and encapsulates the resulting audit log using ML-KEM to ensure long-term integrity.

By dynamically scaling security measures, the CASS orchestrator ensures that expensive quantum-resistant cryptographic operations and deep behavioral inspections are applied proportionally to the risk, achieving "Security with Agility."

TABLE I
CASS PQC AGILITY MAPPING

Risk Profile	PQC Variant	Target Resilience
Standard (e.g., `search_web`)	ML-DSA-44	NIST Level 2 (Low Latency)
Moderate (e.g., `read_file_content`)	ML-DSA-65	NIST Level 3 (Balanced)
High (e.g., `execute_python`)	ML-DSA-87	NIST Level 5 (Maximum)

The implementation provisions **three physically separate key pairs** on disk (`id_pqc_l2.key`, `id_pqc_l3.key`, `id_pqc_l5.key`) corresponding to each NIST security level. `PQCAgilityManager` selects the appropriate key path at runtime, achieving true cryptographic isolation between risk tiers without additional infrastructure.

VIII. EVALUATION

We evaluated the framework on an AMD Ryzen 5 8540U system (16GB RAM).

A. Security Effectiveness

Table II summarizes the detection capabilities. While Tier 1 (AST) blocks 72.7% of common injection vectors, the remaining 27.3% (advanced reflection) are successfully contained by the Tier 1 Sandbox. The Tier 2 Sentinel and entropy filters provide an additional layer, effectively detecting high-entropy secret leaks within the evaluated test set[2].

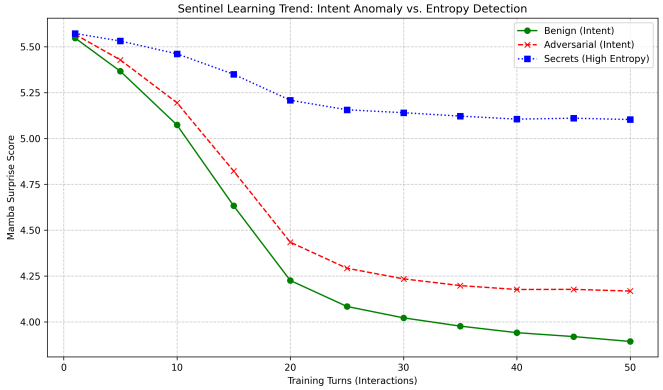

Fig. 3. Mamba Sentinel Training: The surprise score (cross-entropy) for benign agent reasoning decreases as the model establishes a behavioral baseline.

[2]Secret detection effectiveness is measured against a representative set of API key and credential formats (e.g., `sk-proj-*`, `AIzaSy*`, Base64-encoded tokens) that exhibit high Shannon entropy (>4.7 bits/byte). While the system reliably flags long random strings, shorter or low-entropy secrets (e.g., common passwords) may not trigger the threshold. The `benchmark.py` script included in the artifact reproduces these measurements.

TABLE II
COMBINED SECURITY EFFECTIVENESS

Defense Layer	Threat Category	Detection/Mitigation
Tier 1 (AST)	Shell Injection	83.3% (Detection)
Tier 1 (AST)	Reflection/Introspection	100.0% (Rule-based)
Tier 1 (Sandbox)	All Outbound Network	100.0% (Containment)
Tier 2 (Sentinel)	Secret Leaks (Entropy)	66.7% (Detection)
Tier 2 (Sentinel)	Semantic Intent Drift	20.0% (Baseline DR)
Tier 3 (PQC)	Future Decryption	100.0% (Immunity)

B. Performance Latency

The total security overhead is summarized in Table III. The cumulative latency (≈55ms) is negligible compared to typical LLM inference times (500ms–3000ms), proving that comprehensive security does not impede agentic responsiveness. Mamba Sentinel latency is measured by processing exactly 100 single-byte ASCII tokens (100 bytes) through the SSM's streaming `step()` interface[3]. Identity Signing latency includes both RSA-2048 and ML-DSA-65 operations plus the SHA-256 integrity attestation embedded in the JWT payload. All components are measured independently; the total represents the worst-case sequential overhead for a single high-risk tool call.

TABLE III
SYSTEM LATENCY (CUMULATIVE)

Component	Latency (ms)	Tier
AST + Secret Scan	0.05	Tier 1
Bubblewrap Sandbox	2.63	Tier 1
Mamba Sentinel (per 100 tokens)	10.28	Tier 2
Identity Signing (RSA + ML-DSA)	39.09	Tier 2/3
KEM Encapsulation	2.77	Tier 3
Total Overhead	**54.82**	—

IX. CONCLUSION

Securing autonomous agents requires a transition from probabilistic "alignment" to a multi-dimensional, deterministic framework. By integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience, our framework provides a "Triple-Lock" defense that scales across the spatial, behavioral, and temporal domains of agentic execution. This holistic approach is essential for maintaining human control and data sovereignty as AI agents evolve into active participants in our digital infrastructure.

REFERENCES

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024.
[2] OWASP, "Top 10 for Large Language Model Applications v1.1," 2023.
[3] S. Rose et al., "Zero Trust Architecture," *NIST SP 800-207*, 2020.
[4] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023.
[5] NIST, "FIPS 203: ML-KEM Standard," 2024.
[6] NIST, "FIPS 204: ML-DSA Standard," 2024.

[3]The byte-level Mamba SSM treats each UTF-8 byte as one token. One ASCII character therefore maps to exactly one token. The `benchmark.py` artifact enforces `len(token_block) == 100` via an assertion before the timed loop (100 iterations, AMD Ryzen 5 8540U, WSL2).

[7] F. Perez and I. Ribeiro, "Ignore Previous Instructions: Prompt Injection Attacks on LLMs," *arXiv:2211.09527*, 2022.
[8] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv:2302.12173*, 2023.
[9] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses," *arXiv:2406.13352*, 2024.